

1. What You'll Learn in This Section

In this lesson on data preparation, you'll learn to:

- Explain why generalization and a proper train-test-validation split matter before modeling.
- Identify features, the target variable, and whether a column is numerical or categorical.
- Apply pandas techniques to explore data, handle missing values and duplicates, convert data types, and summarize columns.
- Build new features from existing columns, including a ratio metric and a boolean feature.

2. Detailed Explanation

Why data preparation matters: overfitting, generalization, and the ML pipeline

A model that scores very well on training data but noticeably worse on test data suffers from overfitting — it performs much better on data it has already seen than on new data. A model that performs consistently well on both training and test data is said to generalize well. For example, a model scoring 95% on training data and 92% on test data generalizes better than one scoring 99% on training data but only 65% on test data. The second model has essentially memorized the training data instead of learning patterns that carry over to new data.

Getting to a model that generalizes well depends on a full pipeline of steps, always in this order:

Data preparation — the focus of this lesson — sits right in the middle of this pipeline (cleaning, EDA, and feature engineering), and everything downstream depends on getting it right.

Splitting data: train-test-validation split

Before modeling, the feature vectors (X) are separated from the target variable (y). The `train_test_split` function takes four parameters: X, y, `test_size`, and `random_state`.

- `test_size` sets what percentage of the data goes into the test set; the remainder becomes the training set.
- `random_state` makes the "random" split reproducible: using the same value regenerates the exact same split every time the code runs.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

`train_test_split` only produces two sets at a time. To also get a validation set (used to tune the model before final testing), split in two levels. First divide the full dataset into a training set and a temporary set. Then divide the temporary set further into a validation set and a test set.

Building a practice dataset

To make every technique checkable by hand, a small 10-row practice dataset was built with pandas and NumPy. The snippet below is shortened for readability — each column actually holds 10 matching values, one per student:

```
import pandas as pd
import numpy as np
# Each list below is abbreviated with "..." for readability;
# every column holds 10 matching values in the full dataset.
df = pd.DataFrame({
    "student_id": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    "gender": ["male", "female", "male", "female", "male", ...],
    "city": ["Delhi", "Mumbai", "Pune", "Delhi", "Mumbai", ...],
    "age": [...],          # some values intentionally left blank (NaN)
    "hours_studied": [...],
    "attendance": [...],   # in percentage
    "previous_marks": [...],
    "placement": ["yes", "no", "yes", "no", "yes", ...],
})
```

The dataset has 8 columns: `student_id` (unique identifier), `gender`, `city` (Delhi, Mumbai, or Pune), `age` (some missing values), `hours_studied`, `attendance`, `previous_marks`, and `placement` (yes/no). NaN stands for "Not a Number" and represents a missing value in the data.

Defining the problem: features vs. target variable

A problem statement rarely states features and the target explicitly — they must be decoded from context. Here, a college placement team wants to predict, from a student's data, whether that student will get placed. So:

- Target variable: `placement` — the variable the model is trying to predict.
- Features: `student_id`, `gender`, `city`, `age`, `hours_studied`, `attendance`, `previous_marks` — the input variables used to make that prediction.

Watch out for: `student_id` sits among the feature columns, but it is not a real feature. It is just a unique key for each row and carries no predictive information, so it should not actually be used when modeling.

Numerical vs categorical columns

In this dataset, age, hours_studied, attendance, and previous_marks are numerical columns — they hold measurable quantities with many possible values. gender and city are categorical columns — columns with a limited, fixed set of distinct values.

A common misconception is that "categorical" simply means "textual." The correct rule is that a column is categorical if it has a very limited set of distinct values, regardless of whether it is text or numbers. A name column is textual but not categorical, because it can take on so many different values that grouping by it isn't meaningful. Conversely, a purely numeric column can still be categorical if it only takes a small number of distinct values. Examples include zip code, month number (limited to 1–12), and year of study (limited to 1, 2, 3, or 4).

Whether a column is categorical or numerical is data-driven and intuitive, not something to memorize from the column's data type alone.

Exploring the data (EDA basics)

The most fundamental exploratory checks, applied in order:

```
df.head()    # first 5 rows, to see what the data looks like
df.shape     # (10, 8) -> 10 rows and 8 columns
df.columns   # list of all column names in order
df.info()    # data type + non-null count per column
df.describe() # statistical summary of numerical columns
```

- df.head() shows the first five rows so you get a feel for the data's shape and content.
- df.shape returned (10, 8) — 10 rows and 8 columns.
- df.columns lists all column names in their existing order.
- df.info() reports the non-null count and data type per column. Here, every column showed 10 non-null values except age, which showed 8 (2 missing entries). Columns appeared as integer, float, or object dtype — pandas' label for string/text columns.
- df.describe() gives statistics but only for numerical columns. Applying it to student_id produces meaningless statistics, since student_id isn't a real feature. For the meaningful numerical columns, describe() reports count (non-null rows — 8 for age), mean (average), std (standard deviation — how much data deviates from the mean), min/max, and the five-number-summary percentiles.

Handling missing values

First detect missing values:

```
df.isnull()    # cell-by-cell True/False; True means the cell is null
df.isnull().sum() # count of nulls per column
```

`df.isnull()` marks each cell True if null and False otherwise; `df.isnull().sum()` aggregates this into a per-column count. In this dataset, only age had nulls — 2 of them.

There are exactly two strategies for missing values: drop the rows/values, or fill (impute) them with a relevant value. Dropping is compared to the "ostrich mechanism" — an ostrich facing an oncoming storm is said to put its head in the ground, thinking the problem has disappeared. The storm is still there even though the ostrich can't see it. Similarly, dropping rows with missing data doesn't solve the underlying issue; it just removes data and loses information.

Dropping isn't always bad, though. If only a tiny fraction of a column is missing (well under 1%), dropping those rows has no real impact. But if a meaningfully larger portion is missing (around 10% or more), dropping is a poor choice, and imputing is preferred.

When filling a numerical column, the natural instinct is the mean (average), since it summarizes the whole column. But the mean is sensitive to outliers, so when a column has outliers, the median is a better choice, since it isn't influenced by extreme values.

- **Age example:** age has naturally limited outliers (realistic ages go up to 100–110, not 1000), so filling with the mean is reasonable:

```
average_age = df["age"].mean()      # computed as 20.75 in this dataset
df["age"] = df["age"].fillna(average_age)
```

20.75 isn't a "real" whole-number age, but it can be read as roughly "20 and three-quarters" — a reasonable interpolated value.

- **Salary example:** for a monthly salary column mostly in the 10,000–20,000 range but with one unusually large outlier, the median is the better choice, since the mean would be pulled upward by that outlier.
- **Spotting an outlier** doesn't always require a box plot — basic analytical intuition about the data is often enough.

Not every missing value should be filled with a statistical measure, either. If a name field were missing, filling it with the most frequent name (the mode) wouldn't make sense — a domain-specific placeholder like "Unknown" is better. Imputation is the general term for replacing missing data with any substituted value (mean, median, mode, or a domain-specific placeholder), as opposed to dropping rows.

Handling duplicate rows

Detect duplicates with:

```
df.duplicated()      # True/False per row; only later occurrences are marked True
df.duplicated().sum() # total count of duplicate rows
```

`df.duplicated()` checks each row against previous rows — if two rows are identical, only the second (later) occurrence is marked `True`; the first occurrence isn't counted. `.sum()` totals duplicate rows overall, without breaking the count down by which value is repeated.

Calling `df.drop_duplicates()` by itself does not modify the original DataFrame — by default `inplace=False`, so it returns a new, filtered DataFrame instead. There are two correct ways to use it:

Approach	Code	Effect
In-place modification	<code>df2.drop_duplicates(inplace=True)</code>	Modifies <code>df2</code> directly; original data is gone for good. Rarely used in industry.
Reassignment (preferred)	<code>df3 = df2.drop_duplicates()</code>	<code>df2</code> stays unchanged; the duplicate-free result is stored in <code>df3</code> .

Watch out for: mixing both approaches, e.g. `df4 = df2.drop_duplicates(inplace=True)`. When `inplace=True` is used, the method modifies the DataFrame directly and returns nothing — so `df4` ends up empty (`None`). The `inplace` parameter controls this: `True` modifies and returns nothing; `False` (the default) returns a new DataFrame and leaves the original untouched. This same reassignment pattern — keeping the original safe and storing the modified result separately — applies to any data-modifying pandas operation, not just dropping duplicates.

Data type conversion

A column's data type can be explicitly changed with `astype()`:

```
df["age"] = df["age"].astype(int)
```

Applying this to `age` converts a value like 20.75 to the integer 20. This isn't a meaningful loss of information — it mirrors how people naturally state age in everyday life. Someone who is 20 years and 9 months old would still say they're 20, not round up to 21. Truncating the decimal here reflects that same convention.

Aggregation, value counts, and the Pareto principle

Standard pandas aggregate methods can be called directly on a column:

```
df["attendance"].mean()    # average attendance of the class
```

```
df["previous_marks"].max() # highest previous marks scored by any student
df["hours_studied"].mean() # average number of hours studied
value_counts() counts how many times each unique value occurs in a column:
```

```
df["hours_studied"].value_counts()
```

This shows, for example, that a value like 5 occurs twice, 4 occurs twice, and so on. Setting `normalize=True` converts these counts into fractions/percentages instead of raw counts:

```
df["gender"].value_counts(normalize=True)
```

Applied to gender, this showed 6 males and 4 females in raw counts — or 60% male and 40% female normalized. Applied to city, the split was Delhi 40%, Mumbai 30%, and Pune 30%.

This normalized view becomes especially useful with large datasets containing many rare categories. Consider a country-level dataset where the US contributes 30% of the data, India 20%, China 20%, the UK 15%, and the remaining countries contribute tiny slivers (0.1%, 0.05% each). Rather than analyzing every rare category, an analyst might keep only the categories that together cover roughly 90% of the data and treat the rest as noise. This connects to the Pareto principle (the 80-20 rule): a small proportion of categories or causes accounts for a large majority of the outcome.

Group-based analysis with groupby

`groupby` splits data into groups based on a column's values so an aggregation can be computed per group. It answers questions phrased as "for each X, what is the average of Y":

```
df.groupby("city")["hours_studied"].mean()
```

This showed students from Delhi studying an average of 4.7 hours, versus 4.53 hours for Mumbai. A similar `groupby` on gender and attendance:

```
df.groupby("gender")["attendance"].mean()
```

showed females averaging 95% attendance versus males averaging 84.6%.

The general pattern has three steps. First, find the "for each ____" phrase to identify the grouping column. Second, identify the column to aggregate (which sits outside the `groupby` clause). Third, apply the aggregation function (`.mean()`, or another as needed).

Feature engineering

Two new columns were engineered from existing ones:

- Study efficiency — previous marks divided by hours studied, approximating how efficiently a student converts study time into marks:

```
df["study_efficiency"] = df["previous_marks"] / df["hours_studied"]
```

A student scoring high marks with relatively few study hours is considered more efficient than one scoring the same or lower marks with many more hours. The efficient student would likely perform even better with additional hours. This metric is imperfect and not linear; it exists to demonstrate the concept of engineering a feature, not as a production-ready metric.

- High attendance (a boolean feature) — built directly from a comparison expression, with no if/else needed:

```
df["high_attendance"] = df["attendance"] >= 85
```

Because the comparison itself evaluates to True or False per row, assigning it directly to a new column produces the boolean feature automatically. (A threshold of 85 was chosen to show a mix of True and False outcomes.) Once this boolean feature exists, it can be used to compute the fraction of students with high attendance versus those without, and similar boolean features can be built for other columns.

3. Key Takeaways

This lesson covered the core building blocks of data preparation, from splitting data correctly through cleaning it and engineering new features.

- Generalization — performing consistently well on training and test data — is the goal; a proper train-test-validation split and a defined pipeline support it.
- A problem statement's features and target variable must often be inferred from context, and not every feature column (like an identifier) is actually useful for prediction.
- Whether a column is categorical or numerical depends on how many distinct values it has, not on whether it's text or numbers.
- Missing values can be dropped or imputed (mean, median, mode, or a domain-specific value); duplicates need careful inplace vs. reassignment handling to avoid losing data.
- `value_counts()`, `groupby`, and engineered features (ratios, boolean thresholds) turn a cleaned dataset into one that's ready for modeling.

Mental model: Think of data preparation as tidying a workspace before starting a real project. You check what's there (EDA), fix what's broken or missing (cleaning), and build the tools you'll actually need (feature engineering) before the real work of modeling begins.